MAKERERE  UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

CSC1102: STRUCTURED AND OBJECT-ORIENTED PROGRAMMING

PRACTICAL EXAM PROJECT

STUDENT MANAGEMENT SYSTEM

PROGRAMMING

GROUP 5

PROJECT DOCUMENTATION

	NAME	REG No. NUMBER
01	NAKIRIMA JOAN	26/U/31007/PSA
02	TANETA CATHERINE	26/U/03655/PSA
03	MUWANGA ABDUL MALIK	26/U/03579/PSA
04	EWUONZI GILBERT OBIA	26/U/29537/PSA
05	OLANYA COLLINS	26/U/03630/PSA

GITHUB REPOSITORY

https://github.com/Gilbert-Obia/CSC1102-Student-Management-System

1. Project Overview

The Student Management System is a simple desktop application developed in Java using the Swing framework. It is designed to help a user register student records, view all registered students, search for an individual student using a student ID, and delete a student record.

The project demonstrates introductory Java programming and object-oriented design through three main classes: Student, StudentManager, and StudentManagementGUI. Student records are held in an array while the application is running. The application also uses a local text file named students.txt to save records and load them again when the application starts.

1.1 Project Objectives

- Provide a simple graphical interface for managing basic student records.
- Apply object-oriented programming concepts through classes, objects, constructors, encapsulation and inheritance.
- Demonstrate arrays, methods, conditional statements, loops and input validation.
- Use basic file handling to retain student records between application sessions.

1.2 Intended Users and Scope

The application is intended as a small educational student-record management example. It supports one local user operating the desktop application. It is not designed as a full institutional student information system.

2. Main Features

Feature	What it does	How it is used
Register student	Creates a student record and adds it to the array after validation.	Enter the ID, name, programme and year, then select Register Student.
Search student	Finds a record by student ID.	Enter the ID and select Search Student.
Display all	Shows all currently registered student records.	Select Display All.
Delete student	Removes a matching record and closes the empty position in the array.	Enter the ID and select Delete Student.
Save and load records	Writes records to students.txt and loads them at startup.	Saving occurs after registration or deletion; loading occurs when the manager is created.
Input validation	Checks required fields and the year-of-study value.	Empty fields, non-numeric years and years outside 1 to 7 are rejected.

3. Classes and Their Responsibilities

The application is divided into three classes. Each class has a defined responsibility so that student data, record-management operations and the graphical interface are kept separate.

Class	Main attributes / components	Purpose and important methods
Student.java	studentId; fullName; programme; yearOfStudy	Represents one student. Its constructor initialises the details. Getters retrieve values, setters update values, and toString() provides a readable record.
StudentManager.java	Student[] students; studentCount; FILE_NAME	Manages the collection of student objects. Includes addStudent(), searchStudent(), displayAllStudents(), deleteStudent(), saveStudents(), loadStudents() and getStudentCount().
StudentManagementGUI.java	StudentManager manager; text fields; output area; buttons	Builds the Swing window and handles user actions. Its constructor sets up the interface and action listeners. The main() method starts the application.

4. Object-Oriented Programming Concepts

Concept	Application in this project
Classes and objects	Student is the blueprint; each registered student is an object created from that class.
Constructors	Constructors initialise Student, StudentManager and StudentManagementGUI objects.
Encapsulation	Student attributes are private and are accessed or updated through public getter and setter methods.
Inheritance	StudentManagementGUI extends JFrame and inherits window functionality from Swing.
Object interaction	StudentManagementGUI holds a StudentManager object and calls its methods to manage records.

Design note

The GUI uses StudentManager through an object reference. This is interaction between objects (a has-a relationship), while the explicit inheritance example is StudentManagementGUI extends JFrame.

5. How the Classes Interact

The classes work together when a user registers a student. The GUI collects and validates the information, creates a Student object, and passes that object to StudentManager for storage.

1. User interface	The user enters the student ID, full name, programme and year of study in StudentManagementGUI.
2. Object creation	The GUI calls the Student constructor to create a Student object using the entered details.
3. Record management	The GUI passes the object to StudentManager.addStudent(). The manager checks for a duplicate ID and available array capacity.
4. Array storage	If the checks pass, the object is placed in the next available position in the Student[] array and studentCount increases.
5. File storage and feedback	The manager saves the records to students.txt. The GUI displays a success message and clears the input fields.

5.1 Other Operations

- Search: the GUI passes the entered student ID to searchStudent(); the manager checks registered records and returns the matching Student object or null.
- Display: the GUI calls displayAllStudents(); the manager loops through the registered array entries and returns formatted text.
- Delete: the GUI passes the ID to deleteStudent(); the manager finds the record, shifts later entries one position to the left, reduces the count and saves the updated records.

6. Programming Constructs and Validation

Construct / technique	Use in the application
Arrays	Student[] stores up to 100 Student object references.
For loops	Used to search through records, display them, check duplicate IDs and shift records during deletion.
If statements	Check empty fields, valid year ranges, duplicate IDs, search results and deletion results.
Exception handling	NumberFormatException handles a year value that cannot be converted to an integer; IOException is handled during file operations.
Swing event handling	Action listeners connect button clicks to registration, searching, displaying and deletion methods.

7. Data Storage and File Handling

The system uses an array for in-memory record management and a local text file for persistence. The file is named students.txt and is created in the application working directory when records are saved.

- When StudentManager is constructed, it creates the Student array and calls loadStudents() to read previously saved records, if the file exists.
- After a successful registration, saveStudents() writes the current records to students.txt.
- After a successful deletion, saveStudents() rewrites the file with the remaining records.
- The file is read one line at a time. Each line stores the student ID, full name, programme and year of study separated by the | character.

8. Testing Summary

The application was tested during development, and the recorded outcomes were successful for the following functional and input-validation cases.

Test case	Expected behaviour / outcome
Register a student	A valid student record is added and a confirmation message is shown.
Search for an existing student	The matching student details are displayed.
Register more than one student	Multiple Student objects can be stored and displayed.
Delete an existing student	The record is removed and the remaining records are displayed.
Submit empty fields	Registration is rejected with a message requesting all details.
Enter a non-numeric or out-of-range year	The input is rejected with an explanatory message.
Register a duplicate student ID	The duplicate is rejected.
Search for or delete an unknown ID	The application reports that the student was not found.
Restart after saving records	Previously saved records are loaded from students.txt.

9. Scope and Limitations

- The array has a fixed capacity of 100 student records.
- Records are stored locally in a text file; the application does not use a database or synchronise records between computers.
- The application is a simple single-user educational project and does not include authentication or network access.
- The current GUI provides registration, search, display and deletion; it does not provide a separate edit-student screen.

10. Conclusion

The Student Management System is a working Java Swing application that applies three cooperating classes, an array of Student objects, constructors, encapsulation, inheritance, methods, loops, conditional statements, input validation and basic file handling. It meets the scope of a simple beginner-level practical project.

GitHub repository: https://github.com/Gilbert-Obia/CSC1102-Student-Management-System

Student Management System Window

